Лабораторна робота №3 — Повний гайд до захисту

Предметна область: SportsBooking (Бронювання спортивних майданчиків)
Склад системи: API Gateway (порт 5000) + Auth + Catalog + Booking у Docker Compose

Суть роботи: Ми перетворили шлюз `ApiGateway` (порт `5000`) із простого маршрутизатора, зробленого в ЛР1–2, на **єдині захищені вхідні двері системи без бізнес-логіки**: додали складений ендпоінт екрана деталей майданчика (паралельний виклик `CatalogService` + `BookingService` через `Task.WhenAll`), задали таймаути 2 с зі свідомою обробкою збоїв (часткова відповідь `"unavailable"` або `503 ProblemDetails`), перенесли перевірку JWT-токена на шлюз із примусовим видаленням підроблених `X-User-Id`, закрили зовнішні порти сервісів у Docker (`expose`), підключили кеш `IMemoryCache` (TTL 30 с) для спільних даних каталогу з миттєвою інвалідизацією при змінах, налаштували `Rate Limiting` (`429` + `Retry-After`) і довели роботу **18 інтеграційними тестами**.

🗣️ **30 секунд, щоб відкрити розмову з викладачем (прочитай на початку захисту):**
«Шлюз на 5000 — єдині вхідні двері. Складений запит паралельно бере майданчик із Catalog і мою активну бронь із Booking. Таймаут — 2 с. Якщо Booking не відповів, шлюз віддає 200 і статус `unavailable`, а якщо Catalog — то 503. Токен перевіряється на шлюзі, клієнтські заголовки `X-User-Id` примусово стираються. Порти внутрішніх сервісів у Docker закриті через `expose`. Кеш увімкнено лише для каталогу, ключ без користувача, TTL 30 с, скидається миттєво після PUT/POST/DELETE. Ліміти: логін — 10/хв за IP, бронь — 10/хв за юзером, загальний — 100/хв, анонім — 30/хв. Тести: 18 зелених».

Розділ 1. Карта коду — куди дивитися і що показувати

Функціонал	Файл у проєкті	Що саме показати викладачу
Складений ендпоінт	`src/ApiGateway/Controllers/VenueDetailsController.cs`	Маршрут `GET /api/gateway/venues/{venueId}/details`, паралельний виклик `await Task.WhenAll(venueTask, bookingTask)`, обробка 404/503/unavailable.
Власний DTO шлюзу	`src/ApiGateway/Dtos/VenueDetailsDtos.cs`	`VenueDetailsAggDto(Venue, MyActiveBooking, BookingStatus)` — формат екрана клієнта, який належить шлюзу, та статус `GwStatus`.
Клієнти шлюзу + таймаут 2 с	`src/ApiGateway/Clients/CatalogGwClient.cs` `src/ApiGateway/Clients/BookingGwClient.cs`	У `Program.cs` (рядки 205–221): `client.Timeout = TimeSpan.FromSeconds(2)`; у клієнтах перехоплення `TaskCanceledException` → `GwStatus.Unavailable`.
Ендпоінт активної броні	`src/BookingService/Controllers/BookingsController.cs`	Новий метод `GET /api/bookings/my-active?venueId=...` (рядки 93–104): повертає `200 OK` з бронюванням або `204 NoContent`, якщо броні немає.
Перевірка JWT і захист X-User-Id	`src/ApiGateway/Program.cs` (рядки 88–124, 305–326)	`AddJwtBearer` на шлюзі. Рядки 307–308: `Headers.Remove("X-User-Id")` стирає підробку клієнта і підставляє справжній `sub` з перевіреного токена.
Закриті порти сервісів	`docker-compose.yml`	У сервісів `auth` (5001), `catalog` (5002), `booking` (5003) стоїть `expose` замість `ports`. Назовні відкритий тільки шлюз `5000:5000`.
Кеш каталогу + інвалідизація	`src/ApiGateway/Middleware/CatalogCacheMiddleware.cs` `src/ApiGateway/Services/CatalogCacheService.cs`	Кешується лише спільний `GET /api/v1/catalog/**` (TTL 30 с, ключ без юзера, `X-Cache: HIT/MISS`). При `POST/PUT/DELETE` викликається `Invalidate()`.
Rate Limiting (429)	`src/ApiGateway/Program.cs` (рядки 134–199)	Логін: 10/хв за IP; Бронь: 10/хв за `userId`; Загальний: 100/хв за `userId`, анонім: 30/хв за IP. Відповідь `429` + заголовок `Retry-After` (секунди до кінця вікна) + `ProblemDetails`.

Функціонал	Файл у проєкті	Що саме показати викладачу
Єдиний ProblemDetails	`src/ApiGateway/Middleware/GatewayProblemDetailsMiddleware.cs`	Уніфікація всіх помилок шлюзу (401, 404, 429, 503, таймаути) у стандарт RFC 7807 (`application/problem+json`).
Health check + метрики кешу	`src/ApiGateway/Program.cs` (рядки 356–389)	`GET /health` (паралельно опитує всі 3 сервіси → 200 OK або 503 Degraded) та `GET /health/cache-stats` (лічильники `hits` і `misses`).
18 тестів шлюзу	`tests/ApiGateway.IntegrationTests/GatewayTests.cs` `tests/ApiGateway.IntegrationTests/RateLimitTests.cs`	18 інтеграційних тестів (`Scenario11` – `Scenario20`), які покривають усі 6 обов'язкових сценаріїв таблиці з розділу 7 методички.
ADR-0004 та ADR-0005	`docs/adr/0004-gateway-cache-and-rate-limiting.md` `docs/adr/0005-composite-request-venue-details.md`	Архітектурні рішення щодо складу ключа кешу, TTL, алгоритмів лімітування та часткової відмови складеного запиту.

Розділ 2. Розбір коду ЛР3: де змінили, на що, як працює і навіщо

1. Складений ендпоінт і паралельний виклик (Task.WhenAll)

Де лежить: `src/ApiGateway/Controllers/VenueDetailsController.cs` (рядки 123–168)

Що змінили порівняно з ЛР2: У ЛР2 шлюз вміє лише тупо пересилати запити через Ocelot. Ми додали власний контролер шлюзу, який сам паралельно звертається до двох сервісів і склеює екран для клієнта.

```
// 1. Запускаємо обидва виклики ОДНОЧАСНО (не чекаємо завершення першого перед стартом другого!)
var venueTask = _catalogClient.GetVenueAsync(venueId, correlationId, ct);
var bookingTask = _bookingClient.GetMyActiveAsync(venueId, userId, bearerToken, correlationId, ct);

await Task.WhenAll(venueTask, bookingTask); // Чекаємо, поки обидва завершаться паралельно

var venueResult = venueTask.Result;
var bookingResult = bookingTask.Result;

// 2. Якщо майданчика з таким ID взагалі немає в каталозі -> повертаємо 404 ProblemDetails
if (venueResult.Status == GwStatus.NotFound)
    return NotFound(new ProblemDetails { Status = 404, Title = "Venue not found", Detail = $"Venue {venueId} not found."
});

// 3. ПОВНА ВІДМОВА: якщо впав головний сервіс (CatalogService) -> повертаємо 503 ProblemDetails на весь запит
if (venueResult.Status == GwStatus.Unavailable)
    return StatusCode(503, new ProblemDetails { Status = 503, Title = "Catalog service unavailable" });

// 4. ЧАСТКОВА ВІДПОВІДЬ: якщо впав другорядний сервіс (BookingService) -> віддаємо майданчик, але явно пишемо
"unavailable"
var bookingStatus = bookingResult.Status switch {
    GwStatus.Ok          => "active",      // Активна бронь знайдена
    GwStatus.NotFound    => "none",        // Сервіс відповів, що активної броні у юзера немає (204 NoContent)
    GwStatus.Unavailable => "unavailable", // Сервіс бронювань лежить або перевищив таймаут 2 с!
    _ => "none"
};

return Ok(new VenueDetailsAggDto(Venue: venueResult.Data!, MyActiveBooking: bookingResult.Data, BookingStatus:
bookingStatus));
```

Пояснення простими словами (навіщо і як працює):

- **Навіщо `Task.WhenAll` :** Обидва запити летять по мережі одночасно. Якщо Catalog відповідає за 150 мс, а Booking за 200 мс, то клієнт чекатиме лише **200 мс** (за найдовшим), а не 350 мс (якби викликали по черзі через два окремі `await`).
- **Навіщо поле `bookingStatus` :** Викладач вимагає, щоб клієнт відрізняв «даних немає» (`"none"`) від «сервіс впав» (`"unavailable"`). Якби ми мовчки повертали `null` , клієнт подумав би, що його бронювання не існує, і створив би друге таке саме!

2. Таймаути (2 с) та клієнти шлюзу

Де лежить: `src/ApiGateway/Program.cs` (рядки 205–221) та `src/ApiGateway/Clients/CatalogGwClient.cs` (рядки 45–88)
Що змінили порівняно з ЛР2: Додали жорсткий дедлайн 2 секунди на всі вихідні `HttpClient` шлюзу та в `ocelot.json` (`"TimeoutValue": 2000`).

```
// У Program.cs реєструємо типізовані клієнти з таймаутом 2 секунди:
builder.Services.AddHttpClient<ICatalogGwClient, CatalogGwClient>(client => {
    client.BaseAddress = new Uri(catalogUrl);
    client.Timeout = TimeSpan.FromSeconds(2); // Якщо сервіс думає довше 2 с — виклик примусово обірветься!
});

// У CatalogGwClient.cs (і так само в BookingGwClient.cs) ловимо обрив за таймаутом або помилку мережі:
try {
    using var response = await _http.SendAsync(request, ct);
    if (response.StatusCode == HttpStatusCode.NotFound) return new GwResult<VenueDto>(GwStatus.NotFound);
    if (!response.IsSuccessStatusCode) return new GwResult<VenueDto>(GwStatus.Unavailable);
    ...
}
catch (Exception ex) when (ex is HttpRequestException or TaskCanceledException or TimeoutException) {
    _logger.LogError(ex, "CatalogService unavailable for venue {VenueId}", venueId);
    return new GwResult<VenueDto>(GwStatus.Unavailable); // Не валимо шлюз, а повертаємо статус Unavailable!
}
```

Пояснення простими словами: Коли один із сервісів зависає довше ніж на 2 секунди, `HttpClient` викидає `TaskCanceledException` . Наш блок `catch` ловить її і спокійно повертає статус `GwStatus.Unavailable` . Завдяки цьому потоки шлюзу не блокуються назавжди і шлюз ніколи не падає разом із завислим сервісом.

3. Перевірка JWT-токена на шлюзі та захист від підробки X-User-Id

Де лежить: `src/ApiGateway/Program.cs` (рядки 88–123 та 303–326) + `docker-compose.yml`
Що змінили порівняно з ЛР2: У ЛР2 шлюз не перевіряв токен сам. Тепер шлюз валідує JWT на вході, видаляє підроблені клієнтом заголовки `X-User-Id` і записує туди справжній ID із токена. А в `docker-compose.yml` порти сервісів закрили через `expose` .

```
app.UseAuthentication(); // 1. Шлюз перевіряє підпис, термін дії та видавця JWT-токена

app.Use(async (context, next) =>
{
    // 2. ЗАХИСТ: Примусово стираємо заголовки X-User-Id та X-User-Roles, які міг підсунути хитрий клієнт!
    context.Request.Headers.Remove("X-User-Id");
    context.Request.Headers.Remove("X-User-Roles");

    // 3. Якщо користувач пройшов перевірку токена — витягуємо його справжній ID з клейма "sub"
    if (context.User.Identity?.IsAuthenticated == true)
    {
        var uid = context.User.FindFirstValue("sub") ?? context.User.FindFirstValue(ClaimTypes.NameIdentifier);
        if (!string.IsNullOrWhiteSpace(uid))
            context.Request.Headers["X-User-Id"] = uid; // Передаємо вниз перевірений ID!
    }
    await next();
});
```

Пояснення простими словами: Чому це критично важливо? Якби ми не робили `Headers.Remove("X-User-Id")` , будь-хто ззовні міг би надіслати запит із заголовком `X-User-Id: 1` і вкрасти дані адміністратора. А оскільки ми стираємо клієнтський заголовок і закрили порти сервісів (`expose: 5001, 5002, 5003` у Docker), ніхто не може обійти шлюз напряму.

4. Локальний кеш каталогу (IMemoryCache) та миттєва інвалідизація при змінах

Де лежить: `src/ApiGateway/Middleware/CatalogCacheMiddleware.cs` та `src/ApiGateway/Services/CatalogCacheService.cs`
Що додали в ЛР3: Кешування спільних даних каталогу на 30 секунд і скидання кешу після `POST/PUT/DELETE`.

```
public async Task InvokeAsync(HttpContext context)
{
    // 1. ПРАВИЛО БЕЗПЕКИ: Кешуємо ТІЛЬКИ спільний каталог (/api/v1/catalog). Персональні маршрути проходять повз!
    if (!context.Request.Path.StartsWithSegments("/api/v1/catalog")) {
        await _next(context);
        return;
    }

    // 2. ІНВАЛІДИЗАЦІЯ: Якщо адмін змінює каталог (POST, PUT, DELETE) і сервіс відповів успіхом (200..299)
    var method = context.Request.Method;
    if (HttpMethods.IsPost(method) || HttpMethods.IsPut(method) || HttpMethods.IsDelete(method)) {
        await _next(context);
        if (context.Response.StatusCode >= 200 && context.Response.StatusCode < 300)
            _cacheService.Invalidate(); // Миттєво скидає весь кеш каталогу через CancellationTokenSource.Cancel()!
        return;
    }

    // 3. Будуємо ключ кешу БЕЗ користувача: "GET:/api/v1/catalog/items?category=Football&page=1"
    var cacheKey = BuildCacheKey(context.Request);
    if (_cache.TryGetValue(cacheKey, out CachedHttpResponse? cached) && cached is not null) {
        _cacheService.RecordHit();
        context.Response.Headers["X-Cache"] = "HIT"; // Віддаємо миттєво з RAM шлюзу без виклику сервісу!
        await context.Response.Body.WriteAsync(cached.Body);
        return;
    }

    // 4. Якщо в кеші немає (MISS) — викликаємо сервіс і кладемо відповідь у кеш на 30 секунд (TTL)
    _cacheService.RecordMiss();
    ...
    var entryOptions = new MemoryCacheEntryOptions()
        .SetAbsoluteExpiration(TimeSpan.FromSeconds(30))          // TTL = 30 секунд
        .AddExpirationToken(_cacheService.GetExpirationToken());  // Прив'язка до токена скидання!
    _cache.Set(cacheKey, toCache, entryOptions);
    context.Response.Headers["X-Cache"] = "MISS";
}
```

Пояснення простими словами:

- **Що кешуємо:** Тільки `/api/v1/catalog`, бо список спортивних майданчиків однаковий для всіх людей. Мої бронювання (`/api/bookings`) і мій профіль (`/api/v1/users`) **не кешуються взагалі**, щоб один користувач ніколи не побачив чужу бронь!
- **Як працює інвалідизація:** Всі записи в кеші прив'язані до одного «стоп-крана» (`CancellationChangeToken` у `CatalogCacheService`). Коли адмін змінює ціну майданчика через `PUT`, шлюз смикає метод `Invalidate()`, який натискає цей стоп-кран — і всі старі записи миттєво зникають із пам'яті, не чекаючи 30 секунд!

5. Обмеження частоти запитів (Rate Limiting) — захист від спаму і підбору паролів

Де лежить: `src/ApiGateway/Program.cs` (рядки 127–199)
Що додали в ЛР3: Вбудований `Microsoft.AspNetCore.RateLimiting` з розділенням лімітів та відповіддю `429`.

```
builder.Services.AddRateLimiter(options =>
{
    options.RejectionStatusCode = StatusCodes.Status429TooManyRequests;
    options.GlobalLimiter = PartitionedRateLimiter.Create<HttpContext, string>(ctx =>
    {
        var ip = ctx.Connection.RemoteIpAddress?.ToString() ?? "anon";
        var uid = ctx.User.FindFirstValue("sub"); // ID авторизованого користувача з токена

        // 1. Вхід у систему (POST /api/v1/auth/login) — жорсткий ліміт 10 спроб/хв за IP (захист від підбору паролів)
        if (HttpMethods.IsPost(ctx.Request.Method) && path.StartsWithSegments("/api/v1/auth/login"))
            return RateLimitPartition.GetFixedWindowLimiter($"login:{ip}", _ => FixedOpt(10));

        // 2. Створення бронювань (POST /api/bookings) — 10 запитів/хв за ID юзера
        if (HttpMethods.IsPost(ctx.Request.Method) && path.StartsWithSegments("/api/bookings"))
            return RateLimitPartition.GetFixedWindowLimiter($"book:user:{uid ?? ip}", _ => FixedOpt(10));

        // 3. Авторизований юзер — 100 запитів/хв за його UserId; Анонім — 30 запитів/хв за IP
        return uid is not null
            ? RateLimitPartition.GetFixedWindowLimiter($"general:user:{uid}", _ => FixedOpt(100))
            : RateLimitPartition.GetFixedWindowLimiter($"general:ip:{ip}", _ => FixedOpt(30));
    });

    // Коли ліміт перевищено — беремо з лічильника скільки секунд лишилось до кінця вікна (або запасне 60)
    options.OnRejected = async (ctx, ct) => {
        ctx.HttpContext.Response.Headers.RetryAfter =
            ctx.Lease.TryGetMetadata(MetadataName.RetryAfter, out var retryAfter)
                ? ((int)retryAfter.TotalSeconds).ToString() : "60";
        await ctx.HttpContext.Response.WriteAsJsonAsync(new ProblemDetails {
            Status = 429, Title = "Too Many Requests", Detail = "Rate limit exceeded. Please retry later."
        }, ct);
    };
});
```

Пояснення простими словами: Чому авторизованих рахуємо за `uid` (`general:user:5`), а анонімів за `ip` (`general:ip:192.168.1.1`)? Тому що в офісі чи університеті за однією IP-адресою Wi-Fi сидить сотня студентів. Якщо рахувати авторизованих за IP, один спамер заблокує роботу всім іншим. А коли кожен має свій ліміт за `userId`, страждає лише сам порушник. Заголовок `Retry-After` показує, скільки секунд залишилося до кінця поточного хвилинного вікна (наприклад, `37`).

6. Закриті порти в docker-compose.yml, таймаути в ocelot.json та видалення секретів

Де лежить: `docker-compose.yml`, `src/ApiGateway/ocelot.json`, `src/BookingService/Controllers/BookingsController.cs` та всі `appsettings.json`

```
# 1. У docker-compose.yml: внутрішні сервіси мають expose (закриті ззовні!), а ports є ТІЛЬКИ у шлюзу:
  catalog-service:
    expose:
      - "5002"          # Доступний ЛИШЕ всередині мережі Docker (з хоста http://localhost:5002 не відкриється!)
  booking-service:
    expose:
      - "5003"          # Доступний ЛИШЕ всередині мережі Docker
  api-gateway:
    ports:
      - "5000:5000"     # ЄДИНІ вхідні двері назовні!

// 2. В ocelot.json: на кожному маршруті додано QoSOptions з таймаутом 2000 мс (2 секунди):
"QoSOptions": { "ExceptionsAllowedBeforeBreaking": 3, "DurationOfBreak": 5000, "TimeoutValue": 2000 }

// 3. У BookingsController.cs додали метод для складеного запиту шлюзу:
[HttpGet("my-active")]
public async Task<IActionResult> GetMyActive([FromQuery] int venueId, CancellationToken ct = default) {
    var userId = GetCurrentUserId(); // Беремо ID суворо з токена, а не з параметрів!
    var dto = await _bookingService.GetActiveForVenueAsync(userId, venueId, ct);
    return dto is null ? NoContent() : Ok(dto); // 204 якщо броні немає, 200 якщо є
}

// 4. У всіх appsettings.json прибрали секретний ключ (Вимога 6.5: жодних секретів у репозиторії!):
"Jwt": { "Key": "", "Issuer": "SportsBookingAuth", "Audience": "SportsBookingClients" }
```

Пояснення простими словами: Завдяки `expose` у Docker клієнт фізично не може підключитися до портів `5001`, `5002` чи `5003` в обхід шлюзу. А порожній `"Key": ""` в `appsettings.json` гарантує, що в Git-репозиторії немає жодного пароля чи секрету (вони підтягуються з `.env`).

Розділ 3. Команди для консолі Docker та робота зі Swagger

Крок 0. Усі команди для термінала (як запустити, зупинити і перевірити Docker)

Відкрийте термінал у папці `d:\навчання\dotnetLAB\SportsBooking` і використовуйте ці команди:

```
# 1. Запустити всі контейнери у фоні (перед початком показу):
docker compose up -d

# 2. Перевірити, що всі 5 контейнерів працюють (має писати Up):
docker ps

# 3. ЗУПИНИТИ сервіс бронювань з консолі (для Дії 2 — показати часткову відповідь "unavailable"):
docker stop sportsbooking-booking

# 4. УВІМКНУТИ сервіс бронювань назад у консолі (після Дії 2):
docker start sportsbooking-booking

# 5. (Якщо викладач попросить) Зупинити каталог — щоб показати 503 ProblemDetails (повну відмову):
docker stop sportsbooking-catalog
# І одразу повернути назад:
docker start sportsbooking-catalog

# 6. Подивитися логи каталогу (для Дії 4 — довести, що при X-Cache: HIT запит до сервісу не доходить):
docker logs sportsbooking-catalog --tail 20
```

Відкрийте в браузері єдиний Swagger шлюзу: **http://localhost:5000/swagger/index.html**

Крок А. Авторизація в Swagger (отримати і вставити токен)

1. У Swagger шлюзу (`http://localhost:5000/swagger/index.html`) розкрийте зелений блок **POST /api/v1/auth/login**, натисніть **Try it out** і **Execute**:

   ```
   {
     "email": "admin@sportsbooking.com",
     "password": "Admin123!"
   }
   ```

2. Скопіюйте отриманий `accessToken` (без лапок).
3. У Swagger шлюзу вгорі праворуч натисніть зелену кнопку **Authorize** 🔓, вставте токен у поле *Value* і натисніть **Authorize** → **Close**. Замок закриється 🔒.

Крок Б. Виклик складеного ендпоінта (GET /api/gateway/venues/{venueId}/details)

1. Розкрийте синій блок **GET /api/gateway/venues/{venueId}/details** і натисніть **Try it out**.
2. У полі `venueId` введіть `1` і натисніть синю кнопку **Execute**.
3. **Результат:** статус `200 OK`. Шлюз паралельно опитав 2 сервіси і повернув єдиний JSON екрана:

   ```
   {
     "venue": {
       "id": 1,
       "name": "Olympic Arena",
       "address": "вул. Спортивна, 1",
       "pricePerHour": 450.00,
       "category": "Multipurpose"
     },
     "myActiveBooking": {
       "id": 101,
       "venueId": 1,
       "startTime": "2027-06-01T10:00:00Z",
       "hours": 2,
       "totalPrice": 900.00,
       "status": "Confirmed"
     },
     "bookingStatus": "active"
   }
   ```

Крок В. Як викликати наживо кожен із 5 прикладів у Swagger (якщо викладач попросить)

Яку відповідь хоче бачити викладач	Що конкретно натиснути у Swagger і терміналі
1. Успішна відповідь (200 OK) `"bookingStatus": "active"` *(або `"none"` без броні)*	У блоці `GET /api/gateway/venues/{venueId}/details` введіть `venueId = 1` і натисніть **Execute** → поверне майданчик №1 і активне бронювання зі статусом `"active"`. *(Якщо ввести `venueId = 3`, де броні немає → поверне `"myActiveBooking": null, "bookingStatus": "none"`).*
2. Часткова деградація (200 OK) `"bookingStatus": "unavailable"`	1) У терміналі виконайте: `docker stop sportsbooking-booking` 2) У Swagger введіть `venueId = 1` і натисніть **Execute** → виведе `200 OK`, майданчик є, а внизу `"myActiveBooking": null, "bookingStatus": "unavailable"`. 3) Увімкніть назад: `docker start sportsbooking-booking`
3. Повна відмова (503 Service Unavailable) `Catalog service unavailable`	1) У терміналі виконайте: `docker stop sportsbooking-catalog` 2) У Swagger введіть `venueId = 2` (щоб не взяло №1 із кешу!) і натисніть **Execute** → виведе `503 ProblemDetails` (`"title": "Catalog service unavailable"`). 3) Увімкніть назад: `docker start sportsbooking-catalog`
4. Майданчик не знайдено (404 Not Found) `Venue not found`	Нічого зупиняти не треба! Просто у Swagger у поле `venueId` введіть неіснуючий номер `999` і натисніть **Execute** → одразу виведе `404 Not Found` у форматі `ProblemDetails` (`"detail": "Venue with id 999 was not found in catalog."`).
5. Перевищення ліміту (429 Too Many Requests) `Too Many Requests`	У Swagger відкрийте блок `POST /api/v1/auth/login` (де ліміт = 10 запитів на хвилину) і швидко натисніть синю кнопку **Execute 11 разів поспіль** → на 11-й раз виведе `429 Too Many Requests` із заголовком `retry-after` та `ProblemDetails`!

Розділ 4. Покроковий план захисту в Swagger (8 дій перед викладачем за Розділом 9 методички)

Усе виконується прямо в браузері у Swagger шлюзу: `http://localhost:5000/swagger/index.html`

Дія 1. Показати складений ендпоінт: один запит клієнта замість двох

- У Swagger відкрийте синій блок **GET /api/gateway/venues/{venueId}/details**, введіть `venueId = 1` і натисніть **Execute**.
- Покажіть викладачу у відповіді об'єкт `venue` (з каталогу) та `myActiveBooking` (з бронювань).
- **Фраза для викладача:** *«Клієнт робить лише один запит на порт 5000. Шлюз паралельно через Task.WhenAll викликає CatalogService і BookingService та збирає відповідь у власний DTO шлюзу VenueDetailsAggDto».*

Дія 2. Зупинити один сервіс — показати, що шлюз не зависає, а відповідь свідома

- **Як зупинити в консолі:** виконайте в терміналі:

```
docker stop sportsbooking-booking
```

- У Swagger знову натисніть **Execute** у блоці **GET /api/gateway/venues/{venueId}/details** (де `venueId = 1`).
- **Результат:** з'єднання відхиляється миттєво (за мілісекунди), повертається `200 OK`, картка майданчика на місці, а внизу написано: `"myActiveBooking": null, "bookingStatus": "unavailable"`.
- **Фраза для викладача:** *«Сервіс вимкнено, тож відмова миттєва. А таймаут 2 с захищає від завислого сервісу, який мовчить — це перевіряє тест `Scenario12b`. Оскільки головна частина екрана (майданчик) є, ми віддаємо часткову відповідь і явно позначаємо статус бронювання як `unavailable`, щоб клієнт не подумав, ніби броні немає, і не створив дублікат. Якщо ж зупинити CatalogService (`docker stop sportsbooking-catalog`) — шлюз поверне 503 ProblemDetails (повна відмова)».*
- **Як увімкнути назад у консолі:** виконайте в терміналі:

```
docker start sportsbooking-booking
```

Дія 3. Звернутися напряму на порт сервісу в обхід шлюзу — доступу немає!

- Відкрийте нову вкладку в браузері і вставте пряму адресу каталогу (порт `5002` замість `5000`): `http://localhost:5002/api/v1/catalog/items`.
- **Результат:** браузер покаже помилку з'єднання (`ERR_CONNECTION_REFUSED` — «Немає зв'язку із сайтом»).
- **Фраза для викладача:** *«У docker-compose.yml для Auth, Catalog і Booking ми використали expose замість ports. Їхні порти 5001–5003 відкриті лише всередині ізольованої мережі Docker для шлюзу. Назовні торчить тільки порт 5000, тому ніхто не може обійти шлюз і підробити заголовок X-User-Id».*

Дія 4. Повторити запит до каталогу — показати роботу кешу (MISS → HIT)

- У Swagger відкрийте блок **GET /api/v1/catalog/items** → натисніть **Try it out** → **Execute** вперше → внизу в блоці *Response headers* покажіть `x-cache: MISS`.
- Натисніть **Execute** вдруге → у *Response headers* тепер написано `x-cache: HIT`!
- У Swagger відкрийте блок **GET /health/cache-stats** → натисніть **Try it out** → **Execute** → покажіть, що лічильник `hits` виріс.
- **Фраза для викладача:** *«Публічний каталог спільний для всіх, тому ми кешуємо його в IMemoryCache з TTL 30 секунд. Другий запит віддається з оперативної пам'яті шлюзу без звернення до CatalogService».*

Дія 5. Змінити дані в каталозі — показати миттєве скидання кешу (інвалідизацію)

- У Swagger відкрийте помаранчевий блок **PUT /api/v1/catalog/items/{id}** → натисніть **Try it out** (там уже підставлено `id = 1` і нову ціну `550`) → натисніть **Execute**.
- Одразу (не чекаючи 30 секунд!) поверніться до блоку **GET /api/v1/catalog/items** і натисніть **Execute**.
- **Результат:** заголовок знову `x-cache: MISS`, і в списку одразу видно нову ціну `550`!
- **Фраза для викладача:** *«Після успішного PUT/POST/DELETE шлюз автоматично викликає Invalidate() через CancellationChangeToken, тому застаріла ціна миттєво видаляється з кешу».*

Дія 6. Показати, що персональні дані НЕ кешуються (два користувачі, той самий маршрут)

- З токеном першого юзера (`admin@sportsbooking.com`) у блоці **GET /api/gateway/venues/{venueId}/details** (`venueId = 1`) натисніть **Execute** → бачимо його бронювання (`"bookingStatus": "active"`).
- У блоці **POST /api/v1/auth/login** замініть email на другого юзера: `"email": "user2@sportsbooking.com"` (пароль той самий: `"Admin123!"`) → натисніть **Execute** → скопіюйте новий `accessToken`.
- Натисніть вгорі праворуч кнопку **Authorize** 🔒 → **Logout**, вставте токен другого юзера → **Authorize** → **Close**.
- Знову натисніть **Execute** у блоці **GET /api/gateway/venues/{venueId}/details** (`venueId = 1`) → другий користувач бачить `"myActiveBooking": null, "bookingStatus": "none"` (не отримує бронювання першого юзера з кешу!).
- **Фраза для викладача:** *«Спільний майданчик береться з кешу, а персональне бронювання кожного разу запитується наживо для конкретного UserId з токена. Персональні дані не кешуються ні повністю, ні частково».*

Дія 7. Перевищити ліміт запитів — показати 429 з Retry-After

- У Swagger у блоці **POST /api/v1/auth/login** швидко натисніть синю кнопку **Execute 11 разів поспіль**.
- **Результат:** статус `429 Too Many Requests`, заголовок `Retry-After` (наприклад, `37` — скільки секунд лишилось до кінця вікна; `60` є лише запасним) та тіло у форматі `ProblemDetails`.
- **Фраза для викладача:** *«Retry-After показує, скільки секунд лишилось до кінця поточного хвилинного вікна».*

Дія 8. Запустити тести (dotnet test) і показати «злам логіки»

- Виконайте в терміналі:

  ```
  dotnet test
  ```

  Результат: `Passed! - Failed: 0, Passed: 18, Skipped: 0` (усі 18 тестів шлюзу зелені).
- **Якщо викладач скаже: «Зламай рядок у коді й покажи, що тест червоніє» (3 готові варіанти):**
  1. **Зламати захист заголовка X-User-Id:** у `src/ApiGateway/Program.cs` (рядок 307) закоментуйте: `// context.Request.Headers.Remove("X-User-Id");` → запустіть `dotnet test` → тест `Scenario20_ClientSupplied_UserHeaders_AreStrippedAndOverwritten` впаде червоним!
  2. **Зламати скидання кешу:** у `src/ApiGateway/Middleware/CatalogCacheMiddleware.cs` (рядок 49) закоментуйте: `// _cacheService.Invalidate();` → запустіть `dotnet test` → тест `Scenario16b_CatalogCache_MutationInvalidatesCache_SubsequentGetHitsDownstream` впаде червоним!
  3. **Зламати часткову відповідь:** у `src/ApiGateway/Controllers/VenueDetailsController.cs` (рядок 159) змініть `"unavailable"` на `"none"` → тест `Scenario13_CompositeRequest_WhenBookingUnavailable_Returns200WithUnavailableStatus` впаде червоним! Розкоментуйте/поверніть назад — тести знову зелені.

Розділ 5. Готові відповіді на ВСІ 11 контрольних питань + 3 каверзні питання

1. Що таке API Gateway і за що він відповідає?

Відповідь: Це єдині вхідні двері системи для зовнішніх клієнтів (порт 5000). Він відповідає за маршрутизацію запитів до мікросервісів, одноразову перевірку JWT-токена, обмеження частоти запитів (Rate Limiting), кешування спільних публічних даних, встановлення таймаутів та збирання одного екрана з кількох сервісів (складений запит).

2. Чому в шлюзі не має бути бізнес-логіки? Наведіть приклад правила, яке туди просилося, але не належить.

Відповідь: Якщо перенести у шлюз бізнес-правила та розрахунки, він перетвориться на другий моноліт, через який іде весь трафік. *Приклад правила:* «Якщо бронювання триває більше 3 годин — дати знижку 15%» або «Перевірити, чи не перетинається час броні з іншими». Це правило належить суто `BookingService`, який володіє даними бронювань.

3. Чому виклики у складеному ендпоінті виконують паралельно? Як зміниться час відповіді, якщо зробити їх послідовно?

Відповідь: При паралельному виклику через `Task.WhenAll` запити до Catalog і Booking виконуються одночасно, тому загальний час дорівнює **найдовшому** виклику: `max(T1, T2)` (наприклад, `max(200 мс, 250 мс) = 250 мс`). Якщо зробити їх послідовно, час складеться: `200 + 250 = 450 мс`, і клієнт чекатиме майже вдвічі довше.

4. Що станеться зі шлюзом, якщо прибрати таймаути, а один із сервісів зависне?

Відповідь: Один завислий сервіс «покладе» весь шлюз і всю систему (каскадна відмова). Запити в шлюзі накопичуватимуться, усі потоки пулу займуться нескінченним очікуванням, пам'ять вичерпається, і шлюз перестане відповідати навіть на ті маршрути, чиї сервіси працюють нормально.

5. Чому клієнт має відрізняти «даних немає» від «сервіс недоступний»?

Відповідь: Якщо `BookingService` впав, а шлюз мовчки поверне `myActiveBooking: null` (так само, як коли бронювань немає), користувач подумає, що його бронювання не створилося, і натисне кнопку «Забронювати» вдруге — створить дублікат і двічі спише кошти. Тому ми явно повертаємо `bookingStatus: "unavailable"`.

6. Чому перевірку токена винесли на шлюз? За якої умови сервіс може довіряти заголовку X-User-Id?

Відповідь: Щоб перевіряти токен один раз на вході й одразу відсікати неавторизовані запити (401) без навантаження на внутрішні сервіси. Сервіс може довіряти заголовку `X-User-Id` лише за двох умов: 1) до сервісу **неможливо достукатися напряму ззовні** в обхід шлюзу (закриті порти `expose`); 2) шлюз **завжди видаляє** заголовок `X-User-Id`, надісланий клієнтом, і записує туди ID сам із перевіреного токена.

7. Які дані ви кешуєте і чому саме їх? Чому персональні дані не кешують?

Відповідь: Ми кешуємо тільки спільні публічні дані — каталог майданчиків (`GET /api/v1/catalog/**`), бо вони однакові для всіх користувачів і читаються набагато частіше, ніж змінюються. Персональні дані (мої бронювання, мій профіль) кешувати не можна: це критична вразливість безпеки, через яку один користувач може отримати з кешу чужі персональні дані.

8. З чого складається ваш ключ кешу? Як ви обирали TTL?

Відповідь: Ключ складається з HTTP-методу, шляху маршруту та відсортованих параметрів запиту (пагінації, категорії): `GET:/api/v1/catalog/items?category=Football&page=1`. Для об'єкта у клієнті: `venue:{venueId}`. Користувача в ключі немає. **TTL = 30 секунд** — баланс між розвантаженням бази даних при сплесках трафіку та свіжістю даних.

9. Навіщо потрібна інвалідизація, якщо є TTL?

Відповідь: Поки не спливе TTL (30 секунд), клієнти бачили б у каталозі стару ціну майданчика навіть після того, як адміністратор її змінив. Інвалідизація примусово очищує кеш одразу після успішного `POST/PUT/DELETE` через шлюз, тому клієнти бачать оновлену ціну миттєво.

10. Чим fixed window відрізняється від token bucket? Що ви обрали і чому?

Відповідь: Fixed window рахує не більше `N` запитів за фіксований відрізок часу (наприклад, 10 за хвилину), але на межі хвилин може пропустити подвійний сплеск. **Token bucket** має «відро жетонів», яке поповнюється з постійною швидкістю і дозволяє короткі сплески трафіку. Ми обрали `FixedWindowLimiter` з розділенням квот (10/хв на логін, 10/хв на бронь, 100/хв загальний), бо він простий, передбачуваний і надійно блокує підбір паролів.

11. Чому ліміт для анонімних запитів рахують за IP, а для авторизованих — за користувачем?

Відповідь: Про анонімного клієнта шлюз знає лише його IP-адресу. А для авторизованих ліміт рахують за `userId` з токена, бо за одним публічним IP (офісний NAT, університетський Wi-Fi, мобільний оператор) можуть одночасно працювати десятки чесних користувачів. Якби їх рахували за IP, один активний юзер вичерпав би ліміт і заблокував усіх інших.

⚠️ **Каверзне питання 1: «Токен перевіряється один раз на шлюзі, а в тебе і сервіси його перевіряють?»**

Відповідь: Так. Шлюз перевіряє токен на вході і передає вниз сам токен та заголовок `X-User-Id`. А сервіси ще раз перевіряють підпис токена, бо мають свій `Jwt__Key`, і беруть `userId` із токена. Це захист у два шари (Defense in Depth): заголовок `X-User-Id` шлюз очищує й передає, але сервіси додатково довіряють саме токену, тож навіть якби порт випадково відкрили, підробити користувача не вийде.

⚠️ **Каверзне питання 2: «Ти кажеш, що шлюз без бізнес-логіки, а bookingStatus: "active"/"none"/"unavailable" — це не логіка?»**
Відповідь: Ні, це не бізнес-логіка. Це лише технічна позначка стану мережевої відповіді для екрана клієнта (є дані / немає даних / сервіс не відповів). Шлюз не вирішує, чи можна бронювати майданчик, не перевіряє перетин часу і не рахує суми чи знижки — усі ціни та бізнес-правила тримає `BookingService`.

⚠️ **Каверзне питання 3: «У Docker є monolith? Бо в ocelot.json є маршрут на порт 5100».**
Відповідь: Ні, у `docker-compose.yml` моноліту немає. Моноліт залишено в репозиторії з попередніх лабораторних для порівняння, він запускається окремо через `dotnet run` на порту 5100. У ЛР3 в Docker працюють шлюз та 3 мікросервіси: `Auth`, `Catalog` і `Booking`.

✅ **Чекліст перед парою:**

1. Перевір, що в корені проєкту є файл `.env` із заповненими `JWT_KEY` та `MYSQL_ROOT_PASSWORD` (без них Docker не підніметься).
2. Запусти `dotnet test` і переконайся, що всі 18 тестів шлюзу зелені.
3. Один раз прожени кроки в Swagger (`http://localhost:5000/swagger/index.html`) по черзі.